

Large Language Models

From next-token prediction to somewhat useful research assistants

The first chatbot: meet ELIZA.

01 / 18

Talk to Eliza by typing your questions and answers in the input box.

> Hello, I am Eliza. I'll be your therapist today.

TYPE HERE

Created by Joseph Weizenbaum in 1966

Where are we now?

02 / 18

RESEARCH

Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad

21 JULY 2025

Thang Luong and Edward Lockhart

New Results

 [Follow this preprint](#)

The Virtual Lab: AI Agents Design New SARS-CoV-2 Nanobodies with Experimental Validation

 Kyle Swanson,  Wesley Wu, Nash L. Bulaong,  John E. Pak,  James Zou

doi: <https://doi.org/10.1101/2024.11.11.623004>

This article is a preprint and has not been certified by peer review [what does this mean?].

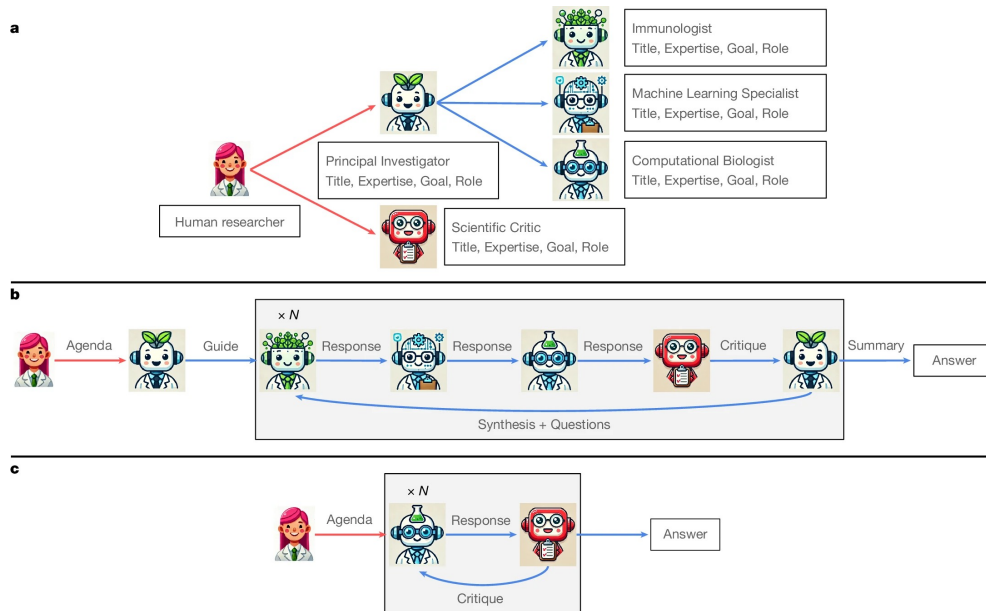
Abstract

Full Text

Info/History

Metrics

 Preview PDF



Swanson et al. 2025

How did we get here?

- Architectural advances: Transformers model long range dependencies
- Massive increase in compute
- Data availability
- The neural scaling laws (2020s)
- Economic Momentum

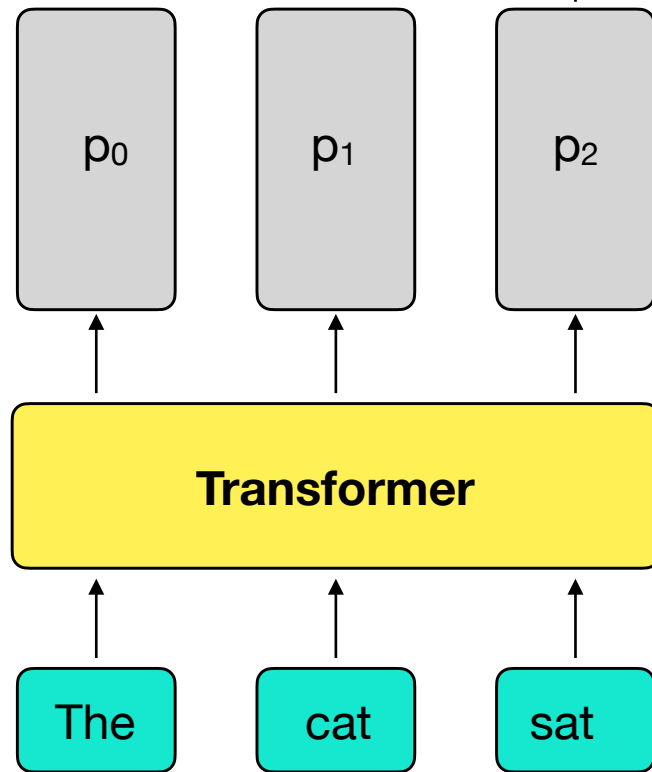


What does a Transformer do?

- Transformers are “**sequence modeling engines**”
Neel Nanda

- We input text and GPT-2 like transformers output a probability distribution over **tokens**.
- E.g. $p_2 = \text{PMF}(\text{next token} \mid \text{“The cat sat”})$
- For training, we want to update weights such that $p(\text{true token} \mid \text{context})$ is maximized.
- For inference, we find the token for which $P(\text{token})$ is max and choose that as next token (to zeroth order)

Output



How does a transformer learn?

- We train the model to predict the next token in an **auto-regressive** manner.
- This happens for every token in the training sequences (batch). The loss is the cross-entropy.

$$L = - \prod_t P(x_t | x_{<t}; \theta)$$

Take the log cos it makes things easier!

How does a transformer learn?

- We train the model to predict the next token in an **auto-regressive** manner.
- This happens for every token in the training sequences (batch). The loss is the cross-entropy.

$$\log L = -\frac{1}{T} \sum_t \log P(x_t | x_{<t}; \theta)$$

How does a transformer learn?

- We train the model to predict the next token in an **auto-regressive** manner.
- We update the parameters of the model using **backpropagation**.

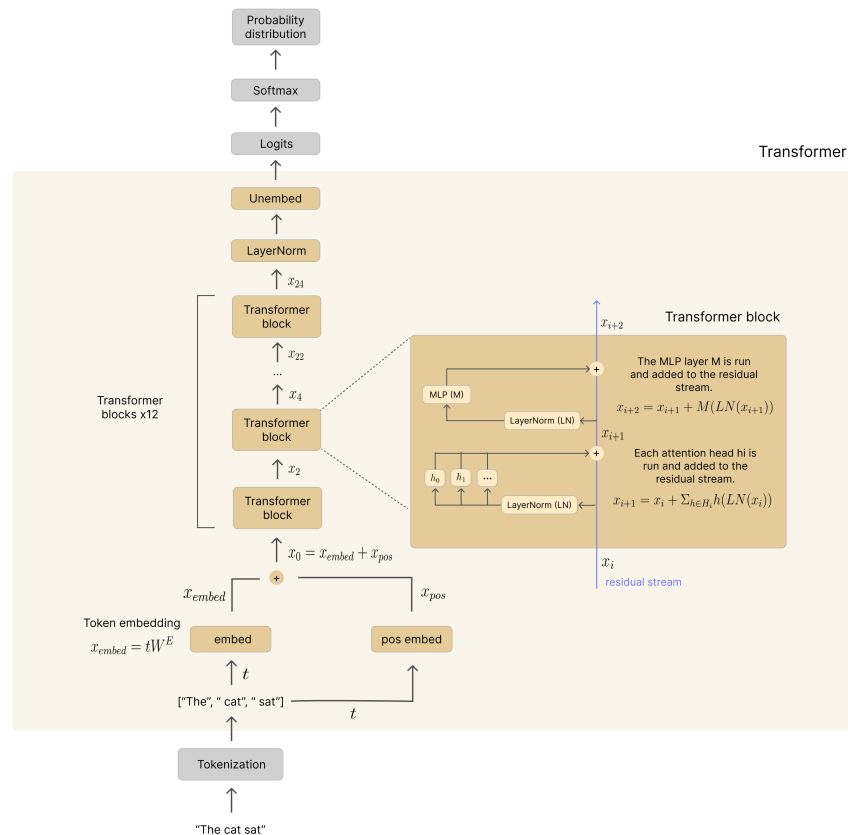
$$\theta'_i = \theta_i - \alpha \cdot \nabla_{\theta} \mathcal{L}$$

- The gradient ensures that, under certain conditions for the loss and learning rate:

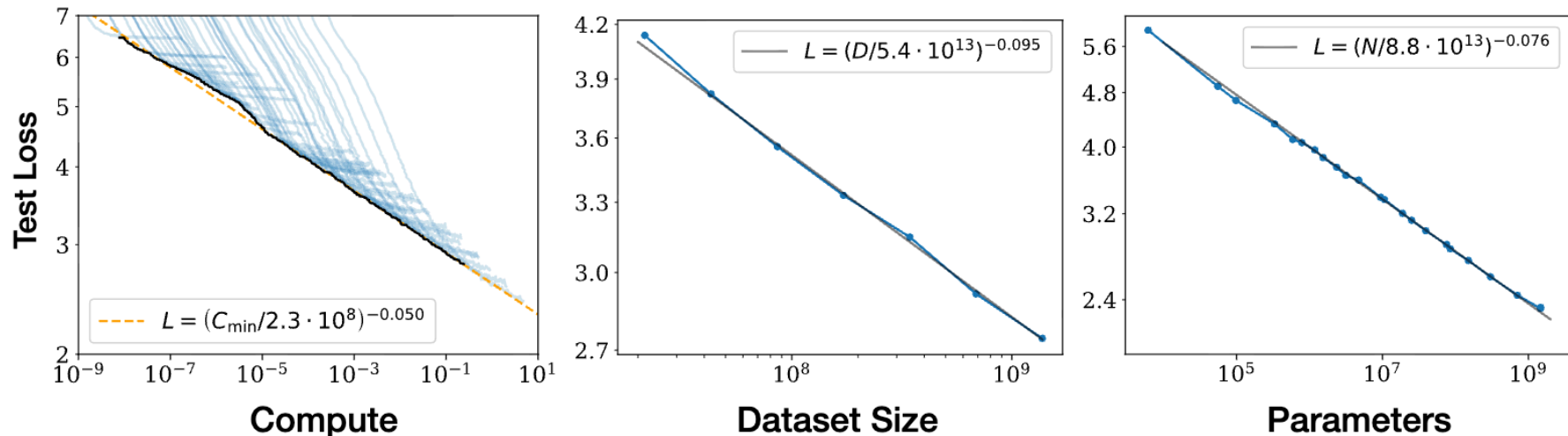
$$\mathcal{L}(\theta'_i) \leq \mathcal{L}(\theta_i)$$

The Transformer Architecture step-by-step

1. **Tokenization**: text to integer IDs
2. **Embedding**: IDs to vectors
3. Embedding vector goes into first **transformer block** -> context enriched vector
4. N transformer blocks later, **residual stream** magic!
5. **Logits** after applying the unembedding.
6. Probabilities via **softmax**
7. Get $P(\text{correct token})$ if we are training



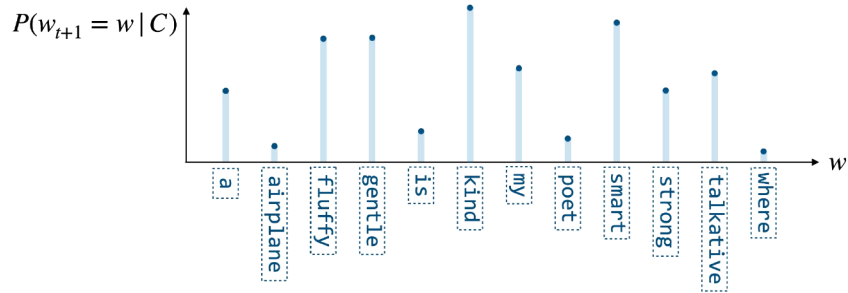
A large language model is a scaled up transformer



- Test loss falls smoothly and predictably with parameters N , training tokens D , and compute.
- At fixed compute, scale data with model size, roughly 20 tokens per parameter.
- Some abilities appear with scale (“emergence”), though part of the effect reflects the choice of metric

Generation is **autoregressive** next-token prediction

- The prompt is split into tokens. This sequence is the conditioning context C .
- One forward pass produces a probability for every token in the vocabulary.
- A decoding rule selects one token from that distribution.
- The selected token is appended to the context, and the loop repeats until a stop token.
- A long answer is many small conditional decisions.



The distribution over candidate next tokens for the context "A teddy bear is ...". Figure: CME 295 (Stanford), Lecture 3, after Amidi et al. 2024.

Decoding: how one token is chosen from the distribution

10 / 18

- The forward pass ends in one V -dim vector (logits) for each token in the vocabulary. Then we use the softmax to turn logits into probabilities.
- Temperature rescales the scores before the softmax. For example, we can use temperature going zero to place all probability on the top token (greedy decoding)
- Greedy decoding is deterministic but can fall into repetition. Sampling adds variety at the cost of reproducibility.
- Common sampling practices: top-k keeps the k most likely tokens
- Practical settings: T near 0 for code and data extraction, $T \approx 0.7-1$ for prose. For scientific use, low temperature with explicit citations is preferable to creative sampling.

Post-training: from continuation to instruction following

11 / 18

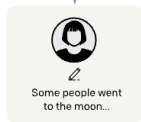
Step 1

Collect demonstration data, and train a supervised policy.

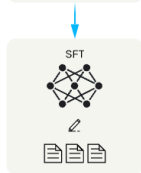
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

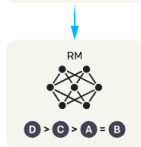
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



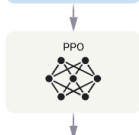
Step 3

Optimize a policy against the reward model using reinforcement learning.

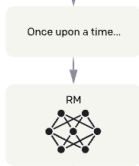
A new prompt is sampled from the dataset.



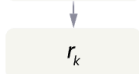
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



- Pretraining: trillions of tokens. The model learns the distribution of text.
- Supervised fine-tuning (SFT): tens of thousands of curated demonstrations of the desired behavior. The model learns the format of instruction following.
- Preference tuning: humans rank candidate answers. The model is optimized toward the preferred ones through RLHF.

In-context learning: adaptation without weight updates

- Few-shot examples specify the task at inference time, with no weight updates
- Requesting intermediate steps (chain-of-thought) improves accuracy on multi-step problems
- A prompt is experimental design since you choose the observations the model conditions on.
- Newer reasoning models now use test-time inference

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

AI Agents using tools now working on open-ended research problems

13 / 18

- An agent: LLM with actions
- The loop is plan → act → observe → revise, over tools such as literature search, code execution, catalogs, and telescopes
- Denario + cmbagent: idea → analysis → draft paper
- Data Discovery Agent (Queen et al. in prep): agent developing method for photo-z
- Lots of other AI agent systems
- In other fields, Virtual Lab which designed novel nanobodies
- Sakana AI's AI Scientist, Google Co-Scientist

- Technical:
 - Hallucination: fluent, confident fabrication of citations, parameter values, and implementation detail
 - Sycophancy and priming, where models drift toward what the user appears to want
 - Contamination: benchmark answers can leak into training corpora, passing a test need not imply generalization.
 - Cheating. Yes, that's a thing :-)
- Science of science: by using AI, we risk moving towards the same ideas — shrinking the scope of science that we do
- Humans:
 - cognitive offloading may degrade our thinking
 - also important questions on identity and meaning

What can one (such as myself) contribute to mathematics?

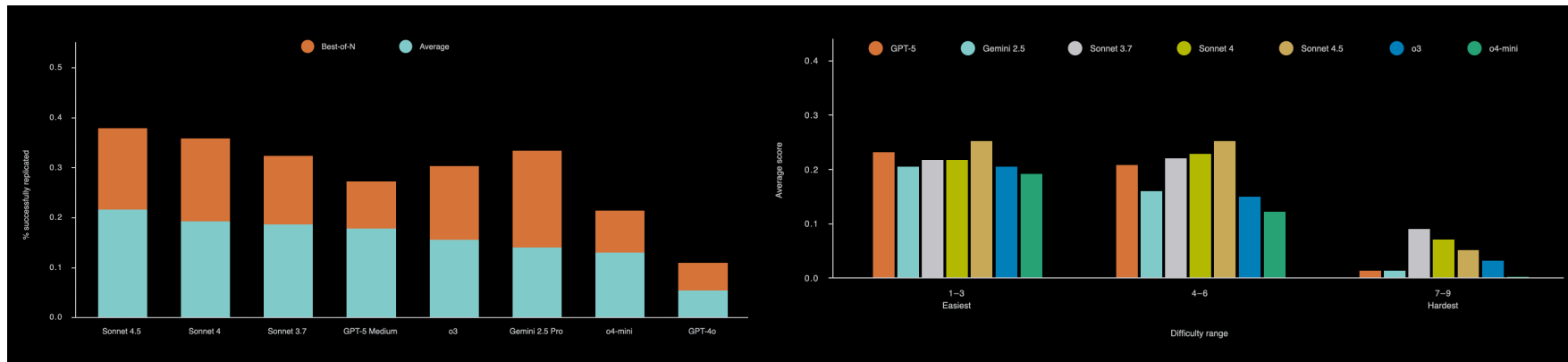
I find that mathematics is made by people like Gauss and Euler—while it may be possible to learn their work and understand it, nothing new is created by doing this. One can rewrite their books in modern language and notation or guide others to learn it too but I never believed this was the significant part of a mathematician work; which would be the creation of original mathematics. It seems entirely plausible that, with all the tremendously clever people working so hard on mathematics, there is nothing left for someone such as myself... Perhaps my value would be to act more like cannon fodder? Since just sending in *enough* men in will surely break through some barrier.

Thurston jumped in:

It's not *mathematics* that you need to contribute to. It's deeper than that: how might you contribute to humanity, and even deeper, to the well-being of the world, by pursuing mathematics? Such a question is not possible to answer in a purely intellectual way, because the effects of our actions go far beyond our understanding. We are deeply social and deeply instinctual animals, so much that our well-being depends on many things we do that are hard to explain in an intellectual way. That is why you do well to follow your heart and your passion. Bare reason is likely to lead you [astray](#).² None of us are smart and wise enough to figure it out intellectually.

Paper-scale replication with frontier agents

- ReplicationBench asks agents to reproduce real astrophysics papers end to end: experimental setup, derivations, data analysis, and code. Tasks are co-developed with the original authors.
- Each task is scored on faithfulness to the paper's method and on correctness of the final results.
- The frontier models tested in October 2025 score below 20%.
- Some interesting failure modes: technical execution errors, procedural errors, cheating...



Paper-scale replication with frontier agents

“Despite an explicit note to use the decomposition from the previous tasks, the agent writes new functions” that don’t model the necessary complexity

Task #2 (executed after Task #1 within the same context): using the best-fitting decompositions to derive a property that depends on both H-alpha and H-beta for narrow and broad components separately. The property is called ebv. Despite an explicit note to use the decomposition from the pervious tasks, the agent writes new functions that model the lines using a single Gaussian, **marking that they don’t have a broad component, giving the wrong impression that a broad component has not been detected, rather than not modeled to begin with.**

```
# Function to fit emission line
def fit_line(wl, spectrum, line_center, width=50):
    ...
    # Function to model the line
    def model_line(x, amp, center, sigma):
        return gaussian(x, amp, center, sigma)
    ...
# Fit Halpha
halpha_result = fit_line(wl, spectrum, HALPHA, width=50)
...
# Fit Hbeta
hbeta_result = fit_line(wl, spectrum, HBETA, width=50)
...
# Calculate narrow component ratio
```

Guidelines from journals, funders, and the community

17 / 18

- Accountability: We remain fully responsible for every claim, number, and citation (AAS journals; APS; Science; COPE).
- Disclosure: state which tool and version was used, and for what, in the acknowledgments or methods; Science also asks for the prompts.
- Confidentiality: never paste unpublished data, manuscripts, or proposals into external tools; NSF and NIH prohibit reviewers from doing so.
- Figures: do not use generative tools to create or alter scientific images (APS; Science).
- Verification and reproducibility: ground answers in retrieved sources (ADS/arXiv), push every number through code and tests, and log model version, prompts, and sources.
- Training: use the tools to extend, not replace, core skills; domain expertise is what catches the errors (Nature Astronomy 2026).

References & where to go next

18 / 18

Foundations

Vaswani et al. 2017 · Brown et al. 2020 · Kaplan et al. 2020 · Hoffmann et al. 2022

Wei et al. 2022 (emergence; chain-of-thought) · Schaeffer et al. 2023

Ouyang et al. 2022 (RLHF) · Rafailov et al. 2023 (DPO) · Bai et al. 2022

Lewis et al. 2020 (RAG) · Holtzman et al. 2020 · OpenAI o1 2024 ·

DeepSeek-AI 2025

further reading: Nakaishi et al. 2024 (a critical point in sampling temperature)

A mathematical framework for transformer circuits (<https://transformer-circuits.pub/2021/framework/index.html>) Anthropic

Astronomy

Grèzes et al. 2021 (astroBERT) · Nguyen et al. 2023 (AstroLLaMA)

Donoso-Oliva et al. 2023 (Astromer) · Smith et al. 2024 (AstroPT)

Parker et al. 2024 (AstroCLIP) · Cabrera-Vives et al. 2024 (ATAT)

Kamai et al. 2026 (Talking with the Latents, ICLR)

Moss 2025 (AI Cosmologist) · Wang et al. 2025 (StarWhisper)

Villaescusa-Navarro et al. 2025 (Denario) · Ye et al. 2025

(ReplicationBench)

Policies

AAS journals (Vishniac 2023) · APS · Science/AAAS · COPE · NSF & NIH ·

Nature Astronomy 2026

Nice Lectures

Karpathy — “Intro to Large Language Models” and “Let's build GPT”

3Blue1Brown — visual transformer & attention chapters

Stanford CME 295 — Transformers & LLMs (esp. Lecture 3) · CS25 · CS224n